

Quantizing Chronos-2: A Systematic Study of Post-Training Quantization for Time Series Foundation Models

Draft v0.1 — 2026-07-22. Working draft; brackets [TBD] mark results to be filled from results/. Every table below specifies its data source in the *Data provenance map* (Appendix D), which doubles as the tracking contract for the evaluation harness.

Structure modeled on multi-technique quantization studies: *A Comprehensive Evaluation of Quantization Strategies for LLMs* (arXiv:2402.16775), *Quantizing Whisper* (arXiv:2511.08093), and the reporting conventions of GPTQ (arXiv:2210.17323) / AWQ (arXiv:2306.00978), adapted to probabilistic forecasting.

Abstract

Time series foundation models (TSFMs) are increasingly deployed for zero-shot forecasting, yet unlike large language models, nothing is known about how they respond to post-training quantization (PTQ). We present the first systematic study of quantizing Chronos-2, a state-of-the-art 120M-parameter encoder-only TSFM with a direct multi-quantile output head. We evaluate [N] PTQ methods (round-to-nearest, HQQ, NF4, FP8, ONNX/OpenVINO INT8, [...]) across bit-widths from W8A16 to W4A16 [W3/W2] on the 27-task Chronos Benchmark II [and GIFT-Eval], measuring not only point and probabilistic accuracy retention (MASE, WQL/SQL) but also *prediction-interval calibration* (coverage error) and *quantile monotonicity* (crossing rate) — failure modes unique to probabilistic forecasters that aggregate accuracy metrics do not detect. We find that [TBD: e.g., 8-bit quantization is accuracy-neutral ($\leq X\%$ WQL degradation) while 4-bit degrades calibration before point accuracy; the most sensitive modules are TBD]. On [hardware], INT8 reduces model footprint by [X] $\times$ and improves CPU latency by [X] $\times$ at [X]% accuracy retention. We release our evaluation framework, which reproduces published per-task benchmark numbers to $<1e-6$, and all per-series predictions for every variant.

Keywords: time series forecasting, foundation models, post-training quantization, model compression, probabilistic forecasting, calibration

1. Introduction

- TSFMs (Chronos, TimesFM, Moirai, Chronos-2) enable zero-shot forecasting; deployment targets include CPU-only servers, edge devices, and high-throughput batch pipelines where fp32 inference is costly.
- Quantization is the standard efficiency lever for LLMs, but its findings do not transfer directly: Chronos-2 is encoder-only (no autoregressive decode, no KV cache), 120M params (small-model regime where quantization bites harder — arXiv:2409.11055), and emits *21 quantile forecasts* whose calibration and monotonicity matter, not just token accuracy.
- **Terminology note (early, prominent):** in the Chronos literature, "quantization" often refers to Chronos-v1's *input value-binning tokenization*. Chronos-2 has no input binning; this paper concerns *weight/activation quantization* of the network.
- No peer-reviewed work quantizes any Chronos-family model (as of 2026-07). Closest artifacts: a community ONNX-INT8 export (unevaluated on benchmarks) and non-peer-reviewed MLX experiments suggesting int8 is free while int4 damages interval coverage — hypotheses we test rigorously.

Contributions 1. First systematic PTQ study of a probabilistic TSFM: [N] methods $\times$ [M] bit-widths on standard zero-shot benchmarks, with bootstrap CIs on all aggregate claims. 2. Calibration-aware evaluation protocol for compressed probabilistic forecasters: coverage error (MACE) and quantile crossing rate (QCR) alongside WQL/MASE retention; per-series degradation distributions and prediction "flips", not just aggregates. 3. Module-level sensitivity analysis, incl. the first quantization data on *group attention* layers (Chronos-2's mechanism for multivariate/covariate in-context learning). 4. Practical deployment guidance: accuracy/efficiency Pareto frontier on GPU and CPU, with an anchor comparison against smaller fp32 models (quantized-120M vs fp32-28M). 5. Open, reproducible framework: reproduces published per-task benchmark numbers to $<1e-6$; all per-series predictions released for every variant.

2. Related Work

(Condensed from docs/RELATED_WORK.md ; full citations there.)

- **TSFMs & their evaluation:** Chronos (arXiv:2403.07815), Chronos-2 (arXiv:2510.15821), TimesFM, Moirai, TiReX; benchmarks: Chronos Benchmark II, GIFT-Eval (arXiv:2410.10393), fev-bench (arXiv:2509.26468); metrics WQL/SQL/MASE, skill scores & win rates with bootstrap CIs.
- **PTQ methods:** RTN, LLM.int8(), GPTQ, AWQ, SmoothQuant, HQQ, NF4/QLoRA, FP8, rotations (QuaRot/SpinQuant); toolkits: torchao, bitsandbytes, ONNX Runtime, OpenVINO/NNCF, Quanto.
- **Quantization evaluation methodology:** comprehensive evals (arXiv:2402.16775, arXiv:2409.11055); calibration-set effects (arXiv:2311.09755); beyond-accuracy distributional fidelity / flips (arXiv:2407.09141); encoder(-decoder) precedent from Whisper (arXiv:2511.08093): dynamic > static activations, symmetric per-channel > asymmetric per-tensor, degradation concentrates on hard inputs.
- **TSFM compression:** structured pruning (arXiv:2505.23195), distillation/DistilTS (arXiv:2601.12785), edge QAT for small TS transformers (arXiv:2407.11041, 2408.16495). Weight quantization of TSFMs: none published — the gap this paper fills.

3. Background: Chronos-2

Facts that shape the quantization design space (Table 1):

Table 1 — Model architecture summary. | Property | Value | ---|---| | Backbone | Encoder-only transformer (T5-derived), 12 layers, d_model 768, d_ff 3072, 12 heads, RoPE | | Input | Patch size 16, per-instance standardization + arcsinh, residual-net embedding (no tokenization) | | Attention | Alternating time attention / group attention (multivariate & covariate ICL) | | Output | Direct quantile head, 21 levels {0.01–0.99}, up to 1024 steps in one forward pass | | Params / precision | 119.5M, shipped fp32 (478 MB safetensors) | | Inference | Non-autoregressive; no KV cache; compute-bound at batch |

Implications: (i) weight-only quantization buys footprint but not necessarily latency (no memory-bound decode); (ii) W8A8/FP8 with integer/FP8 GEMMs is the throughput lever; (iii) the quantile head is tiny — keeping it high-precision is nearly free; (iv) known precision sensitivity in the input-normalization path (Chronos PR #197) motivates keeping normalization/embedding in fp32 by default.

4. Method: Quantization Design Space

Factors (full grid in Appendix B):

- **Bit-width / format:** bf16 (baseline), FP8-E4M3, W8A16, W8A8-dynamic, W4A16 (g=32/64/128), [W4A8, W3, W2 exploratory].
- **Method family:** RTN (torchao), HQQ, NF4 (bitsandbytes), LLM.int8(), ONNX-RT dynamic INT8, OpenVINO NNCF static INT8 [stretch: GPTQ / SmoothQuant via llm-compressor, AWQ-style scaling].
- **Granularity:** per-tensor vs per-channel weights; group size; static vs dynamic activations.
- **Calibration** (for calibrated methods): set size {32, 128, 512, 1024 series} × domain (in-domain / cross-domain / synthetic) × ≥3 seeds.
- **Module scope:** which modules quantize (attention QKV/O, FFN wi, FFN wo, patch embed, quantile head; time- vs group-attention layers; encoder depth thirds); default high-precision set: norms, input normalization path, residual adds, softmax.

All variants share one evaluation path (model_transform hook); the fp32 model is the reference for every retention number; bf16 is the deployment baseline. fp16 is excluded (documented T5-lineage overflow pathology).

5. Experimental Setup

- **Benchmarks:** Chronos Benchmark II (27 zero-shot tasks; primary sweep arena); [GIFT-Eval 97 configs for promoted configs; fev-bench for covariate tasks — future tier].
- **Metrics:** per benchmark convention — WQL + MASE (9-quantile grid), plus the full supplementary suite (SQL, MQL, MAE, RMSE, RMSSE, WAPE, SMAPE, MAPE) recorded for every run; aggregation: relative to Seasonal Naive → clipped [1e-2, 1e2] → geometric mean → skill score; pairwise win rates (ties = 0.5); 95% bootstrap CIs (1,000 task resamples). Identical code to fev-bench (we reuse fev.analysis).
- **Quantization-specific:** accuracy retention (metric ratio vs fp32, GM over tasks); MACE + per-interval coverage {0.8, 0.6, 0.4, 0.2}; QCR; per-series ΔWQL distribution; flip rate (tasks/series where variant crosses the Seasonal-Naive boundary or the beats-fp32 boundary); win rate of each variant vs its fp32 parent.
- **Efficiency protocol:** model bytes & effective bits/param (incl. scales — Appendix C); peak GPU memory during eval; batch-1 latency (median/p90 over ≥20 warm reps) and throughput (series/s) at batch {1, 32, 256} with context 2048, horizon 64 (paper convention), on [RTX 2000 Ada Laptop 8 GB, driver 596.58] and CPU [model]; backend/kernel named per row; model load time.
- **Hygiene:** all runs carry git SHA + environment + benchmark-file hash; deterministic inference (direct quantile head, no sampling); dataset fingerprints verified against published reference results; failures logged, never silently dropped.
- **Reproduction gate (passed):** our harness reproduces published per-task Seasonal-Naive MASE/WQL to <1e-6 on Benchmark II; our fp32 Chronos-2 ranks above chronos-bolt-base published anchors ([83]% win rate on dev subset).

6. Results

Campaign v0.2 (2026-07-23) — 18 quantized variants × 27 tasks, full Benchmark II. Complete method × bit-width matrix in results/tables/campaign_master.csv. Retention = geometric-mean WQL(variant)/WQL(fp32) over 27 tasks, 1000-resample bootstrap CI. Headline findings (all confirmed at full scale):

1. **8-bit is free across every method** (WQL retention 0.999–1.013): RTN, HQQ, torchao, bnb, even W8A8 dynamic activations. Method-agnostic.
2. **GPTQ achieves fp32 parity down to 3 bits.** W4-GPTQ 0.996–1.002 (CI includes 1.0); **W3-GPTQ 1.047 [0.988–1.137], still statistical parity**, vs W3-HQQ 1.229 and W3-RTN 1.695. Hessian error compensation moves the accuracy cliff ≥1 bit lower.
3. **Method dominates bit-width.** At 4 bits, WQL retention spans 0.996 (GPTQ) → 1.66 (naive RTN) — a 66-point spread — while a good method costs ≤6% going 8→4 bits.
4. **Calibration degrades before accuracy, and only GPTQ preserves it.** coverage[0.8]: fp32 0.750; W4-GPTQ 0.756 (intact) vs W4-HQQ 0.630, W4-NF4 0.686. QCR (quantile crossing) rises under every method even when accuracy/coverage are perfect (W8-RTN: acc-neutral yet QCR 0.039→0.149) — an accuracy-invisible failure mode that only a probabilistic-structure metric detects.
5. **Tail-concentrated damage.** Mean retention hides worst-case: W3-HQQ averages 1.23 but its worst task is 8.2×; per-series ΔWQL distributions (persisted for every run) are the honest view.

6. **Rigorous negative result:** AWQ-style activation-aware scaling — with *provably exact* folds on this architecture — barely beats plain RTN (W4 1.64 vs 1.66). Error *compensation* works here; activation *scaling* alone does not.
7. **Efficiency (storage-real variants, RTX 2000 Ada):** peak batch-1 GPU memory fp32 493 MB → W8 154–176 MB → W4-NF4 87 MB (5.7×); throughput ~flat (~120 series/s, dequant-at-forward). Weight-only PTQ buys footprint, not speed — the fused-kernel arms (torchao, ONNX-RT) are the speed comparison.

Table 2 — Master accuracy/calibration matrix (Benchmark II, 27 tasks; retention vs fp32, bootstrap CI).

variant	method	weight_bits	eff_bpw	size_MB	WQL_ret	WQL_lo	WQL_hi	MASE_ret	worst_task_WQL	QCR	cov80	parity_with_fp32
w4-gptq-g64	gptq	4	4.2500	477.9000	0.9962	0.9783	1.0185	1.0083	1.2100	0.2237	0.7539	True
w8a16-rtn	rtn	8	8.0343	120.1000	0.9994	0.9970	1.0014	1.0008	1.0120	0.1494	0.7492	True
w8-gptq	gptq	8	8.1250	477.9000	0.9998	0.9904	1.0092	1.0030	1.0550	0.0641	0.7489	True
w8-hqq	hqq	8	9.0000	119.6000	1.0020	1.0001	1.0043	1.0003	1.0300	0.0862	0.7459	False
w4-gptq	gptq	4	4.1250	477.9000	1.0024	0.9846	1.0231	1.0113	1.2020	0.2535	0.7557	True
w8a16-torchao	torchao_int8wo	8	NaN	477.9000	1.0028	1.0003	1.0063	1.0017	1.0420	0.1553	0.7477	False
w8a16-rtn-pt	rtn	8	8.0000	119.6000	1.0065	1.0000	1.0135	1.0105	1.0600	0.2563	0.7085	False
w8a8-torchao-dyn	torchao_int8dyn	8	NaN	477.9000	1.0126	1.0041	1.0244	1.0040	1.1230	0.1553	0.7412	False
w8-bnb-int8	bnb_int8	8	NaN	119.6000	1.0127	1.0021	1.0306	1.0027	1.2150	0.1469	0.7431	False
w3-gptq	gptq	3	3.2500	477.9000	1.0468	0.9881	1.1367	1.0587	2.4210	0.3662	0.7913	True
w6-rtn-sim	rtn	6	NaN	477.9000	1.0599	1.0281	1.1040	1.0452	1.5630	0.3422	0.6430	False
w4-hqq	hqq	4	5.0000	59.8000	1.0600	1.0167	1.1238	1.0265	1.8800	0.4198	0.6302	False
w4-bnb-nf4	bnb_nf4	4	4.1270	59.9000	1.0649	1.0007	1.1486	1.0256	2.4960	0.3955	0.6864	False
w3-hqq	hqq	3	4.2010	47.9000	1.2292	1.0911	1.4806	1.1426	8.2440	0.5886	0.6662	False
w3-awq-rtn	awq_rtn	3	NaN	477.9000	1.5009	1.3133	1.7700	1.4192	4.7730	0.7991	0.4795	False
w4-awq-rtn	awq_rtn	4	NaN	477.9000	1.6389	1.3772	2.0109	1.4088	6.7130	0.5988	0.4814	False
w4-rtn-sim	rtn	4	NaN	477.9000	1.6560	1.3838	2.0534	1.4035	6.9940	0.6105	0.4666	False
w3-rtn-sim	rtn	3	NaN	477.9000	1.6947	1.4665	2.0108	1.6152	5.9610	0.7584	0.3671	False
w2-rtn-sim	rtn	2	NaN	477.9000	4.0824	3.0826	5.5662	3.1077	31.4890	0.5229	0.0601	False
fp32 (ref)	none	32	32.0000	477.9000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0388	0.7500	True

Table 3 — Calibration under quantization. Rows: variant. Columns: MACE; coverage[0.8] / [0.6] / [0.4] / [0.2] (nominal vs empirical); QCR; ΔQCR vs fp32 baseline (fp32 QCR is nonzero on some tasks — e.g. 0.178 on covid_deaths — so deltas, not absolutes). [TBD]

Table 4 — Efficiency. Rows: variant. Columns: bits/param (effective), checkpoint MB, peak GPU mem (eval), batch-1 latency p50/p90 (GPU, CPU), throughput series/s @ batch 256 (GPU, CPU), load time, backend. [TBD after P1 profiler + variants]

Table 5 — Anchors: quantized-large vs fp32-small. chronos2-W8 / chronos2-W4 vs chronos-2-small (28M fp32) and chronos-bolt-small/base fp32: skill, retention, size, latency. Answers the deployment question directly. [TBD]

Figure 1 — Accuracy–efficiency Pareto. WQL skill vs model bytes; WQL skill vs CPU/GPU latency; frontier annotated by method. [TBD]

Figure 2 — Quality vs bit-width curves. WQL/MASE retention and MACE vs effective BPW, one line per method; the degradation cliff. [TBD]

Figure 3 — Per-series degradation. Violin/ECDF of per-series ΔWQL (variant – fp32) per bit-width; flip-rate annotation. Computed from persisted per-series predictions. [TBD]

Figure 4 — Calibration curves. Nominal vs empirical central-interval coverage per bit-width. [TBD]

7. Analysis

- **Module sensitivity (Table 6):** leave-one-module-type-quantized / leave-one-out grids; heatmap module × bit-width of ΔWQL. Expected cliff: FFN down-projection (GLU-outlier precedent); novel data: group-attention vs time-attention sensitivity. [TBD after P4]
- **Calibration-set ablations (Table 7):** size / domain / seed variance for calibrated methods. [TBD]
- **Difficulty slicing (Figure 5):** ΔWQL vs series volatility/seasonality-strength deciles (Whisper precedent: damage concentrates on hard inputs). From persisted predictions. [TBD]
- **Where does the damage live?** decomposition: point error vs interval width vs interval placement; quantile-level breakdown from per-quantile WQL/SQL columns. [TBD]

8. Discussion & Recommendations

[TBD: deployment recipe per target (GPU batch, CPU server, edge); which methods to skip; what transfers from LLM practice and what does not.]

9. Limitations

- Single model family (Chronos-2 120M [+ 28M small]); zero-shot univariate benchmark focus [covariate/multivariate tiers pending]; consumer-laptop hardware (absolute latency is hardware-specific; retention ratios are the transferable claim); PTQ focus (QAT only if W4 fails); pretraining-corpus overlap disclosed as in the Chronos-2 paper.

10. Reproducibility Statement

Code, configs, lockfile, vendored benchmark definitions, reference results, per-task summaries, per-series predictions, and run metadata (git SHA, environment, hardware, benchmark hash) are released. The harness reproduces published per-task baseline numbers to <1e-6 (continuously enforced by `pytest -m network`).

Appendix A — Full per-task result tables

[Generated from `results/raw/*/summaries.csv`]

Appendix B — Variant grid and hyperparameters

[Generated from `configs/quantization/*.yaml`]

Appendix C — Effective bits-per-parameter accounting

BPW = (param_bytes + buffer_bytes incl. scales/zero-points) × 8 / num_params, reported by `chronosquant.models.inspect.model_summary`; cross-checked against checkpoint bytes on disk.

Appendix D — Data provenance map (tracking contract)

Paper artifact	Source
Table 2 (accuracy matrix)	<code>results/raw/<run>/summaries.csv</code> : <code>WQL</code> , <code>MASE</code> , <code>SQL</code> , ... + <code>chronosquant.analysis.leaderboard / retention_table</code>
Table 3 (calibration)	<code>summaries.csv</code> : <code>MACE</code> , <code>QCR</code> , <code>coverage[*]</code> columns
Table 4 (efficiency)	<code>results/profiles/<variant>_<device>.json</code> (latency/throughput/memory) + <code>summaries.csv</code> : <code>model_bits_per_param</code> , <code>model_total_bytes</code> , <code>peak_gpu_memory_mb</code> , <code>model_model_load_time_s</code>
Table 5 (anchors)	same as Tables 2+4 for anchor models + <code>results/reference/chronos_zeroshot/*.csv</code>
Table 6 (sensitivity)	<code>summaries.csv</code> of module-scoped runs (P4 configs)
Table 7 (calibration ablations)	<code>summaries.csv</code> of calibration-sweep runs (P4 configs)
Fig. 1–2 (Pareto, bit-width curves)	Tables 2+4 columns
Fig. 3, 5 (per-series, difficulty)	<code>results/raw/<run>/predictions/<task>.parquet</code> (<code>y_true</code> + point + all quantiles per series/step)
Fig. 4 (calibration curves)	<code>coverage[*]</code> columns
Per-quantile breakdowns	<code>WQL[q]</code> , <code>SQL[q]</code> columns
Environment/hardware statements	<code>results/raw/<run>/run_metadata.yaml</code> (<code>environment</code> , <code>model_card</code> , <code>benchmark_sha256</code>)
Statistical significance	bootstrap CIs from <code>fev.analysis</code> (seed 123, 1,000 resamples)

Anything a reviewer can ask for reduces to these artifacts; if a new metric is needed, it is computable offline from the persisted predictions without re-running any model.